

CS571: Programming Languages

Assignment 6. Due: Wednesday, April 22, 11:59PM

For this assignment, you need to submit your solution as one single file named “hw6.txt” to Brightspace. You may NOT use any of Scheme’s imperative features (assignment/loops). Define auxiliary functions where appropriate. You should use Racket (<https://racket-lang.org>) for your implementation. For all problems, you can assume all inputs obey the types as specified in a problem.

Testing and Auto Grading We may use a script to grade the assignment. So it is important to check the following requirements before you submit: (i) the function names in your submission must match EXACTLY as specified in this assignment; (ii) make all of your function definitions global (i.e., use “define”); (iii) name your submission as “hw6.txt”.

To help you test the code, we provide a *subset* of testsuite that we use for grading. You can debug your code and resubmit any time before the deadline. After the deadline, a final grade will be assigned based on the entire testsuite, including the hidden ones.

Problem 1 [20pt] In lecture, we define Church number $\underline{n}$ as $\lambda f. \lambda z. (f^n z)$, where $(f^n z)$ represents the n -fold composition of function f applied to z , i.e. $(f (f \dots (f z)))$ where f is repeated for n times).

- a) (5pt) Implement a function `funPower`, which takes a function f , an integer n and returns the function f^n . For example:

```
((funPower sqrt 2) 16) ; returns 2
((funPower not 3) #t) ; returns #f
```

- b) (5pt) Implement a function `encode`, which takes a natural number n and returns the church encoding of n . For example, `(encode 2)` should return $\underline{2}$, the function $\lambda f. \lambda z. f (f z)$ in *curried* form.

```
((encode 2) sqrt) 16) ; returns 2
((encode 3) not) #t) ; returns #f
```

- c) (5pt) Implement a function `decode`, which takes $\underline{n}$ (the Church encoding of some natural number n) and returns n . For example:

```
(decode (encode 2)) ; returns 2
```

- d) (5pt) Implement a function `MULT`, which takes two Church numbers $\underline{n_1}$, $\underline{n_2}$ and returns the Church number of $n_1 \times n_2$ (i.e., $\underline{n_1 \times n_2}$). For example:

```
(decode (MULT (encode 2) (encode 3))) ; returns 6
```

Problem 2 [5pt] Define a function `dot`, which takes two lists of numbers and returns the dot product of them. The dot product of two lists of numbers $(x_1 x_2 \dots x_m)$ and $(y_1 y_2 \dots y_n)$ is $\sum_{i=1}^{\min(m,n)} x_i y_i$. If any one of the lists is empty, your implementation should return 0. For example,

```
(dot '(2) '(3)) ; returns 6
(dot '(2) '(3 4)) ; returns 6
(dot '(1 -1 1 -1) '(1 1 1 1)) ; returns 0
```

Problem 3 [5pt] Define a function `in?`, which takes an item and a list, and returns a Boolean value indicating if that item appears in the list. Your implementation should return `#f` if the list is empty. For example,

```
(in? 4 '(1 2 3)) ; returns #f
(in? "abc" '(1 #f "abc")) ; returns #t
(in? 2 '(1 (1 2) 3)) ; returns #t
```

Problem 4 [5pt] Define a function `lstOddSum`, which given a list of numbers returns the sum of numbers at odd positions in the list. Your implementation should return 0 if the list is empty. For example,

```
(lstOddSum '()) ; returns 1
(lstOddSum '(1 2)) ; returns 1
(lstOddSum '(1 2 3)) ; returns 4
```

Problem 5 [20pt] Implement the following functions in Scheme using higher order functions like `foldl`, `filter` and `map`. DO NOT use recursive definition for this problem.

a) (5pt) Define a function `thres`, which takes a lower bound a , an upper bound b and a list of numbers, and returns a new list consisting of numbers in the old list that are $\geq a$ and $\leq b$. For example,

```
(thres 0 1 '(-2 -1 0 1 2)) ; returns '(0 1)
(thres 0.5 1.5 '(-2 -1 0 1 2)) ; returns '(1)
(thres 1 0 '(-2 -1 0 1 2)) ; returns '()
```

b) (5pt) Define a function `trunc`, which takes a lower bound a , an upper bound b and a list of numbers, and replaces all numbers that are $< a$ by a and all numbers that are $> b$ by b . For example,

```
(trunc 0 1 '(-2 -1 0 1 2)) ; returns '(0 0 0 1 1)
(trunc 0.5 1.5 '(-2 -1 0 1 2)) ; returns '(0.5 0.5 0.5 1 1.5)
(trunc 1 1 '(-2 -1 0 1 2)) ; returns '(1 1 1 1 1)
```

c) (5pt) Define a function `leq`, which takes two lists of numbers $(x_1 x_2 \dots x_n)$, $(y_1 y_2 \dots y_n)$ and returns `#t` if and only if $x_i \leq y_i, \forall i$. For example,

```
(leq '(1 2 3) '(2 3 3)) ; returns #t
(leq '(1 0) '(0 1)) ; returns #f
(leq '() '()) ; returns #t
```

d) (5pt) Define a function `dup`, which takes a list and duplicates every element in that list. For example,

```
(dup '()) ; returns '()
(dup '("abc" #t 7 7)) ; returns '("abc" "abc" #t #t 7 7 7 7)
(dup '(1 (2 3))) ; returns '(1 1 (2 3) (2 3))
```
